

A PROJECT REPORT ON

Personality Prediction Using Machine Learning

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Nakul Bhangale
Aryan Jasuja
Vraj Gujrathi

Exam No: B400050337
Exam No: B400050323
Exam No: B400050408



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Personality Prediction Using Machine Learning

Submitted by

Nakul Bhangale

Exam No: B400050337

Aryan Jasuja

Exam No: B400050323

Vraj Gujrathi

Exam No: B400050408

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Dipali D. Kadam** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. D. D. Kadam
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Personality Prediction using Machine Learning**’.*

*We would like to take this opportunity to thank **Prof. D.D. Kadam** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*Our special thanks to our honourable **Dr. P. T. Kulkarni** , Director, PICT for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Nakul Bhangale
Aryan Jasuja
Vraj Gujrathi
(B.E. Computer Engg.)

ABSTRACT

The rise of social media usage has led to a massive growth in user-generated content, which holds valuable insights into individuals' personalities. Analyzing such content can provide an alternative to traditional personality assessments by predicting psychological traits based solely on natural language use. This process, known as personality prediction, involves extracting textual features and mapping them to a recognized personality model. In this project, we adopt the Myers-Briggs Type Indicator (MBTI) — a widely used model that classifies individuals into 16 personality types based on four dichotomies — as the basis for personality classification.

We use a dataset sourced from Kaggle containing MBTI-labeled social media posts, and apply natural language processing techniques such as TF-IDF vectorization, sentiment analysis, and part-of-speech tagging to extract meaningful features. These features are fed into supervised machine learning algorithms, primarily Random Forest and Support Vector Machine (SVM), which are trained to classify each of the four MBTI dimensions independently. The system architecture includes a lightweight graphical user interface (GUI) built using CustomTkinter, enabling real-time personality prediction with user-friendly interaction.

Our experiments demonstrate that the proposed system performs well across all MBTI dimensions, achieving an average accuracy of 83% on unseen test data. Unlike deep learning models that require extensive training resources and large datasets, our approach offers a balance between interpretability, efficiency, and accuracy. The results highlight the potential of language-based personality prediction systems for real-world applications such as digital self-assessment, mental health awareness, and personalized recommendation engines.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	3
1.3	Problem Definition	3
1.3.1	Problem Statement	3
1.3.2	Objectives	3
1.4	Project Scope	4
1.5	Methodologies of Problem solving	4
2	Literature Survey	6
3	Software Requirements Specification	10
3.1	Dataset	11
3.2	Functional Requirements	11
3.3	Non-Functional Requirements	12
3.4	External Interface Requirements	14
3.4.1	User Interfaces	14
3.4.2	Hardware Interfaces	14
3.4.3	Software Interfaces	15
3.4.4	Communication Interfaces	15
3.5	Analysis Models: SDLC Model to be applied	16
4	System Design	17
4.1	System Architecture	18
4.1.1	Modules	19
4.2	UML Diagrams	21
4.2.1	Activity Diagram	21
4.2.2	Use Case Diagram	22
4.2.3	Data Flow Diagrams	23
4.2.4	Sequence Diagram	25
4.3	Algorithm and Methodologies	26

4.3.1	MBTI Dimensions and Trait Comparison	28
5	Project Plan	29
5.1	Project Estimate	30
5.1.1	Reconciled Estimates	30
5.1.2	Project Resources	31
5.2	Risk Management	32
5.2.1	Risk Identification	32
5.2.2	Risk Analysis	32
5.2.3	Overview of Risk Mitigation, Monitoring, Management	33
5.3	Project Schedule	34
5.3.1	Project Task Set	34
5.3.2	Timeline Chart	35
5.4	Team Organization	35
5.4.1	Team structure	35
5.4.2	Management reporting and communication	36
6	Project Implementation	37
6.1	Overview of Project Modules	38
6.2	Tools and Technologies Used	40
6.3	Algorithm Details	42
6.3.1	Random Forest Classifier	42
6.3.2	Support Vector Machine (SVM)	42
6.3.3	Model Workflow Summary	43
7	Software Testing	44
7.1	Type of Testing	45
7.2	Test Cases & Test Results	46
8	Results	47
8.1	Outcomes	48
8.2	Screenshots	50
9	Conclusions	54
9.1	Conclusions	55
9.2	Future Work	55
9.3	Applications	56
10	Appendix	57
10.1	Appendix A	58
10.2	Appendix B	61
10.3	Appendix C	63

List of Figures

4.1	System Architecture Diagram	18
4.2	Activity Diagram	21
4.3	Use Case Diagram	22
4.4	0th Level DFD of System	23
4.5	1st Level DFD of System	23
4.6	2nd Level DFD of System	24
4.7	Sequence Diagram	25
8.1	Landing Window	50
8.2	Login Window	51
8.3	Login Attempt	51
8.4	Registration Window	52
8.5	Main Window	52
8.6	Example 1: Predicted Personality ESTP	53
8.7	Example 2: Predicted Personality INFJ	53
10.1	certificate 1	61
10.2	certificate 2	62
10.3	certificate 3	62
10.4	Plagiarism Report Summary	63

CHAPTER 1

INTRODUCTION

1.1 Overview

Personality analysis has long played a vital role in fields such as psychology, behavioral science, human-computer interaction, and marketing, providing deep insights into individual behavior, decision-making, and social preferences^[1]. Traditional personality assessment methods—such as the MyersBriggs Type Indicator (MBTI)—require self-report questionnaires and formal testing environments. While widely adopted in personal and professional development, these techniques are often time-consuming, subjective, and not scalable for realtime applications^[2].

With the explosion of user-generated content on social media platforms, digital traces of personality are now more accessible than ever. Users express themselves through informal text, emojis, hashtags, and behavior patterns, providing a rich source of data for personality analysis. In fact, earlier studies ^[11, 12, 13, 14, 15, 16] demonstrate that there is a strong correlation between user personalities and their online behavior on social media. This has enabled a shift from manual psychological testing to computational methods for personality inference. Studies have shown that

linguistic cues such as word usage, sentiment, and syntactic patterns correlate strongly with personality traits^{[3],[1]}.

Although a large portion of prior work has focused on the Big Five personality model (OCEAN), recent studies have turned toward MBTI-based classification due to its categorical and interpretable nature. The MBTI framework segments individuals into 16 personality types based on four psychological dichotomies: Introversion/Extraversion (I/E), Sensing/Intuition (S/N), Thinking/Feeling (T/F), and Judging/Perceiving (J/P). This makes MBTI particularly well-suited for supervised machine learning classification tasks^[4].

In this study, we propose a scalable machine learning pipeline to classify users into MBTI types based on features extracted from social media text. Our approach involves advanced natural language processing (NLP) techniques such as lexical normalization, sentiment analysis, and part-of-speech tagging, followed by supervised classification using Support Vector Machines (SVM) and Random Forest algorithms. Unlike deep learning-based

approaches, our framework emphasizes interpretability and performance on smaller, noisy datasets, which are common in real-world social media scenarios.

This research aims to explore the effectiveness of classical machine learning models for personality classification, identify linguistically significant features linked to MBTI traits, and assess how well models generalize across personality dimensions. Potential applications include personalized recommendation systems, user profiling, and mental health diagnostics^[5].

1.2 Motivation

The motivation behind this project is to leverage the vast, untapped potential of social media data to understand human personality. As online interactions increasingly shape modern life, accurately predicting personality traits from digital behavior can revolutionize fields like personalized marketing, mental health assessment, and user experience design. This project aims to bridge computational techniques and psychological theory, providing deeper insights into the link between language and personality.

1.3 Problem Definition

1.3.1 Problem Statement

Text-based personality prediction using social media data through supervised machine learning. Text data is analyzed to extract patterns linked to personality, and models are trained using Random Forest and SVM.

1.3.2 Objectives

The main objectives of the project are as follows:

1. To develop a machine learning-based system capable of predicting MBTI personality types from user-generated text using linguistic features.
2. To predict personality traits based on their social media text input using machine learning models.
3. To implement and compare the performance of supervised learning models (Random Forest and SVM) for accurate and interpretable personality classification.
4. To contribute toward mental health awareness and digital self-assessment by providing individuals with insights into their personality traits using text analysis.

1.4 Project Scope

This project focuses on developing a machine learning-based system to predict an individual's MBTI personality type using textual data from social media. The system leverages natural language processing for feature extraction and uses supervised algorithms like Random Forest and SVM for classification. The scope includes data preprocessing, model training, performance evaluation, and identifying relevant linguistic features tied to personality traits.

1.5 Methodologies of Problem solving

The project follows a structured methodology that begins with data collection and preprocessing. The collected text data was highly unstructured and informal, necessitating extensive cleaning. This involved removing special characters, stopwords, links, and MBTI-type mentions to avoid bias. Lexical normalization, tokenization, and stemming techniques were applied to prepare the data for feature extraction.

Once the data was cleaned, we moved on to feature engineering using various Natural Language Processing (NLP) techniques. Features such as

TF-IDF vectors, part-of-speech (POS) tags, sentiment polarity, and linguistic style markers were extracted. These features were selected for their ability to reflect personality indicators as observed in previous research. Each user's data was then converted into a numerical feature vector representing their language patterns and stylistic tendencies.

The personality prediction problem was decomposed into four binary classification tasks based on MBTI dimensions (I/E, S/N, T/F, J/P). For each task, we trained two different supervised learning models—Support Vector Machine (SVM) and Random Forest. These models were chosen for their robustness, interpretability, and suitability for text classification. Hyperparameters were tuned using grid search and performance was evaluated using metrics such as accuracy, precision, recall, and F1-score.

CHAPTER 2

LITERATURE SURVEY

The application of machine learning techniques for personality prediction has gained significant attention in recent years, driven by the explosion of user-generated content on social media platforms and the need for scalable, real-time personality inference. Traditional personality assessment methods, such as the Myers-Briggs Type Indicator (MBTI), have been widely used in corporate, educational, and psychological contexts, but they often require manual administration and are subject to social desirability bias. To address these limitations, researchers have explored automated personality prediction models using linguistic features and behavioral cues extracted from online data sources.

Golbeck et al. ^[1] were among the first to demonstrate the viability of predicting personality traits from social media data. Their study showed that linguistic and interaction patterns on Facebook could be used to infer Big Five personality traits with significant accuracy. Although their work focused on the OCEAN model, it paved the way for applying similar approaches to MBTI classification.

Plank and Hovy ^[2] introduced a Twitter-based dataset labeled with MBTI types and applied both Support Vector Machines and logistic regression for classification. They found that n-gram-based features and psycholinguistic attributes derived from LIWC (Linguistic Inquiry and Word Count) improved classification performance, particularly for the Introversion/Extraversion dimension.

Park et al. ^[3] explored automatic personality classification using a combination of topic modeling and deep learning. They used Latent Dirichlet Allocation (LDA) to identify latent topics from user-generated content and integrated them with Convolutional Neural Networks (CNNs) to predict MBTI traits. Their model achieved promising results, highlighting the potential of topic-aware personality classifiers.

Verhoeven et al. ^[4] developed TwiSty, a large dataset of over 1.2 million tweets annotated with MBTI types. Using a range of supervised learning models including Random Forests and SVMs, they emphasized the difficulty of generalizing MBTI prediction across diverse user profiles. Their findings also stressed the importance of handling class imbalance, a recurrent chal-

lenge in MBTI classification due to unequal distribution of personality types.

More recent efforts have integrated transformer-based models such as BERT to enhance prediction accuracy. Yamada et al. ^[5] fine-tuned a multilingual BERT model on MBTI-labeled text data and demonstrated that pre-trained language models can capture subtle semantic cues linked to psychological types. Despite their higher computational cost, such models have significantly outperformed traditional bag-of-words approaches in many classification benchmarks.

In parallel, studies like those by Filardi et al. ^[6] have analyzed the interpretability of machine learning models in the MBTI domain. By evaluating feature importance in Random Forests and attention mechanisms in deep learning models, they attempted to correlate linguistic markers with MBTI dichotomies, providing psychological insights in addition to classification results.

Several recent studies have extended personality prediction beyond traditional machine learning by incorporating advanced language models. Mairesse et al. ^[7] explored a wide range of linguistic cues, such as acoustic features and dialogue patterns, and analyzed their correlation with the Big Five and MBTI traits. Their approach demonstrated that combining multiple modalities could improve personality trait detection.

Majumder et al. ^[8] proposed a deep learning approach to personality recognition using RNNs and attention mechanisms. By modeling long-term dependencies in text, their method outperformed traditional SVM-based classifiers on several benchmarks, especially when dealing with informal and context-dependent text.

Sun et al. ^[9] introduced a CNN-based framework to predict personality directly from raw text without manual feature engineering. Their findings highlighted that even relatively shallow neural networks could capture semantic information relevant to personality, especially when trained on social media corpora.

Shen et al. ^[10] leveraged both text and metadata (such as user activity and posting behavior) to improve classification accuracy. By using multi-modal fusion, their model achieved improved results on personality prediction tasks and demonstrated the advantage of incorporating behavioral features alongside linguistic ones.

These studies collectively affirm that personality traits—especially those defined in the MBTI framework—can be inferred using machine learning methods applied to linguistic and behavioral features. However, challenges such as data sparsity, imbalance, and generalizability remain prevalent. Our work extends this research by applying classical yet robust algorithms (SVM and Random Forest) on a carefully preprocessed social media dataset, focusing on model explainability and performance across each MBTI dimension.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

To successfully implement the MBTI personality prediction system, several key components were required including data resources, development tools, libraries, and computational infrastructure. These requirements formed the foundation for the preprocessing, modeling, training, and evaluation stages of the project. The accuracy and efficiency of the final model heavily depended on the quality of the dataset and the effectiveness of the tools used for data handling and machine learning.

3.1 Dataset

For this project, we used the publicly available dataset titled “MBTI Personality Type Dataset” from Kaggle, accessible at <https://www.kaggle.com/datasets/datasnaek/mbti-type>. The dataset consists of over 8,600 rows, where each entry contains a user’s MBTI personality type along with their social media posts. The posts are concatenated into a single string per user, separated by “———”. This dataset is well-suited for supervised learning, as it provides labeled data for all 16 MBTI personality types.

The dataset enabled us to perform text-based feature extraction and train classification models without the need for manual data sourcing. However, it also required extensive preprocessing to remove noise, special characters, and references to the personality types themselves in order to prevent data leakage during model training. The availability of labeled data made it feasible to break the problem down into four binary classification tasks, corresponding to each MBTI dimension.

3.2 Functional Requirements

Functional requirements describe the core features and actions that the system must be able to perform to fulfill its intended purpose. For our personality prediction system, the key functional requirements include:

1. **Data Input Handling:** The system should be capable of loading and reading user text data in the format provided by the Kaggle dataset.

2. Preprocessing Pipeline: It must clean, tokenize, and normalize the input text to prepare it for feature extraction.
3. Feature Extraction: The system should be able to extract relevant linguistic features using NLP techniques such as TF-IDF, sentiment analysis, and part-of-speech tagging.
4. Model Training and Classification: It must support training of classification models (SVM and Random Forest) for each MBTI dimension and be able to make predictions on new/unseen data.
5. Result Display: The system should display predicted MBTI types for users and optionally present model performance metrics such as accuracy and F1-score.

These functionalities form the backbone of the solution and are necessary for the system to perform accurate personality type prediction.

3.3 Non-Functional Requirements

Non-functional requirements refer to the quality attributes of the system, which do not relate directly to its core functions but affect user experience, performance, and reliability. For this project, the non-functional requirements include:

1. Scalability: The system should be capable of handling a moderately large number of user text entries without significant performance degradation.
2. Maintainability: The codebase should be modular and well-documented to allow easy modifications or future enhancements.
3. Interpretability: The models used (particularly Random Forest) should provide interpretable outputs such as feature importance to support analysis and validation.
4. Portability: The solution should run on common computing environments with standard hardware specifications (e.g., Intel i5/i7 with 8–16 GB RAM).
5. Responsiveness: During prediction, the system should respond in real-time or with minimal delay to ensure a smooth user experience.

These requirements ensure that the final product is usable, reliable, and suitable for deployment or further development.

3.4 External Interface Requirements

3.4.1 User Interfaces

The project includes a graphical user interface (GUI) developed using Python's Tkinter library. The GUI serves as the primary point of interaction for users and is designed to be simple, intuitive, and user-friendly. Upon launching the application, users are greeted with a registration and login screen. New users can create an account, while returning users can log in with their credentials.

Once logged in, the user is presented with the main interface where they can input custom text or select a user sample from the dataset for personality prediction. The interface also displays the predicted MBTI type. The GUI design ensures that users with no programming background can easily interact with the system and obtain meaningful personality insights.

3.4.2 Hardware Interfaces

The hardware requirements specify the minimum and recommended specifications needed to run the personality prediction system efficiently. As this project involves natural language processing and machine learning tasks, moderate computational power is sufficient.

- Minimum Requirements:
 1. Processor: Intel Core i5 (8th Gen or later) / AMD Ryzen 5
 2. RAM: 8 GB
 3. Storage: 500 MB of free disk space
 4. Operating System: Windows 10 / Ubuntu 20.04 or later
- Recommended Requirements:
 1. Processor: Intel Core i7 / AMD Ryzen 7 or higher
 2. RAM: 16 GB or more
 3. Storage: SSD with at least 1 GB free space
 4. GPU (Optional): NVIDIA GPU with CUDA support for potential integration with deep learning models in the future

These specifications ensure smooth execution of preprocessing, training, and evaluation tasks without significant performance lags.

3.4.3 Software Interfaces

The software requirements include the programming tools, libraries, and environments needed to implement and execute the project components.

1. Operating System: Windows, Linux, or macOS
2. Programming Language: Python 3.8 or later
3. Libraries and Frameworks:
 - NumPy and Pandas (data manipulation)
 - Scikit-learn (machine learning models and evaluation)
 - NLTK and TextBlob (natural language processing and sentiment analysis)
 - Matplotlib and Seaborn (data visualization)
4. IDE / Development Environment: Spyder IDE / Jupyter Notebook / VS Code
5. Package Manager: pip or conda for installing dependencies
6. Virtual Environment: Anaconda

These tools form the core software stack for the project, enabling data handling, model development, and results visualization in an efficient and organized manner.

3.4.4 Communication Interfaces

Since this project is developed as a standalone application, there are no external network communication requirements. All interactions occur locally between system components such as the data processor, NLP module, and machine learning models.

However, the system can be extended to include communication interfaces in the future. For instance, integration with a web application would require support for HTTP-based APIs to send and receive data. In such a case, libraries like requests or frameworks like Flask and FastAPI could be used to enable communication between the frontend and backend components.

3.5 Analysis Models: SDLC Model to be applied

The Agile Software Development Life Cycle (SDLC) model can be highly beneficial for our project. Agile allows iterative development in small loops, allowing continuous improvements which are based around user/client feedback and the evolving and changing project requirements. This is important in machine learning projects like ours where model adjustments and data pre-processing often require refinement. Agile's flexibility supports rapid changes in algorithms or techniques without major disruptions, ensuring that the best-performing models are identified and applied.

Additionally, Agile promotes collaboration through regular sprints and reviews, allowing team members to integrate their expertise in data science, software engineering, and user experience. This iterative process also helps detect issues early, related to data quality or performance, reducing the risk of significant setbacks. Ultimately, Agile ensures a more adaptive and user-centered approach to developing a personality prediction system that meets stakeholder expectations effectively.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

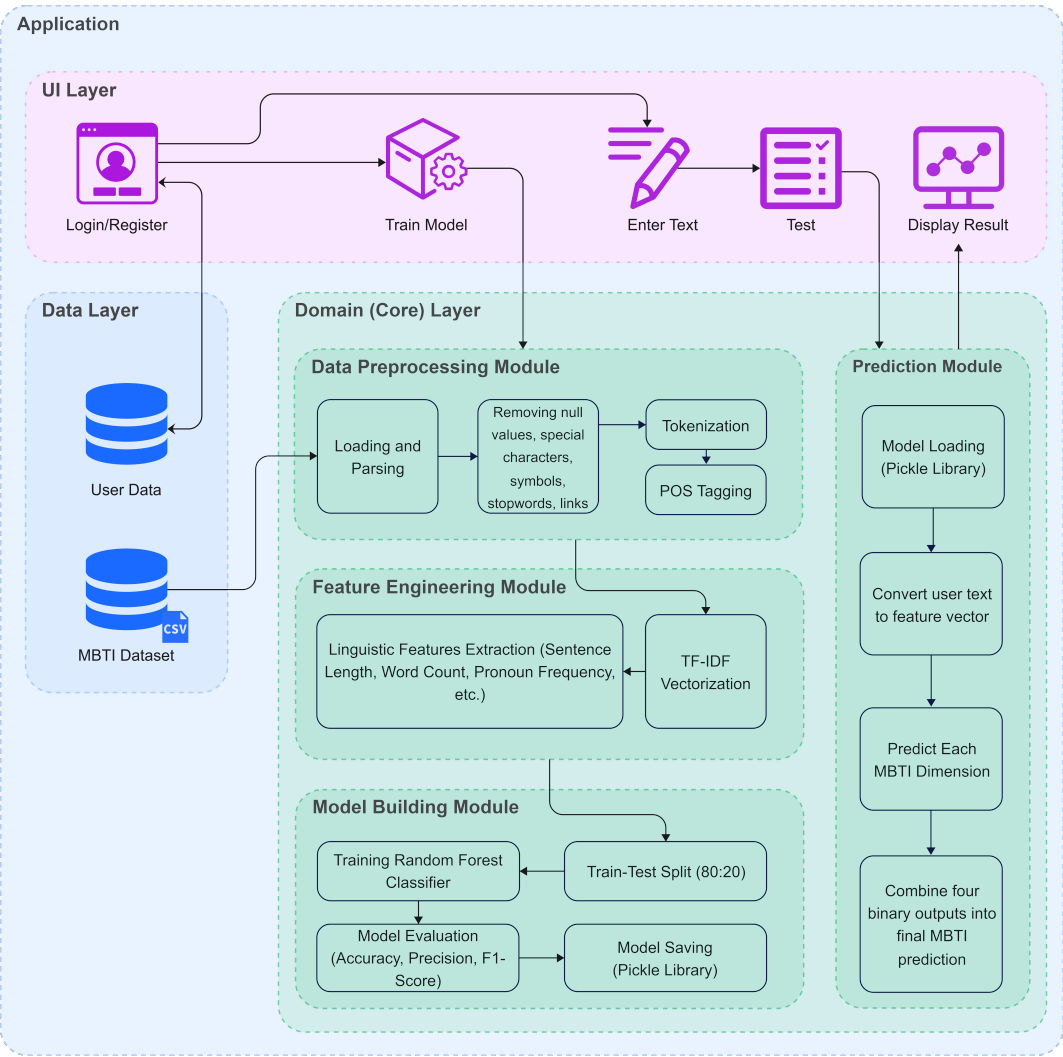


Figure 4.1: System Architecture Diagram

4.1.1 Modules

The system is designed using a modular architecture to ensure maintainability, scalability, and logical separation of responsibilities. Each module plays a specific role in the MBTI personality prediction pipeline:

1. UI Layer

This module is built using CustomTkinter and serves as the graphical user interface of the system. It provides functionality for user registration and login, text input for prediction, and display of the predicted MBTI type. The interface is designed to be simple, intuitive, and visually appealing for ease of use.

2. Data Layer

The data layer is responsible for handling user data and saved models. It includes the SQLite database used for storing registered user credentials securely and managing login sessions. It also handles loading the MBTI dataset from CSV format for model training.

3. Data Preprocessing Module

This module prepares raw input text for analysis by applying various preprocessing techniques. These include lowercasing, removing null values, eliminating special characters and MBTI type mentions, tokenization, stopword removal, and stemming or lemmatization. It ensures that the text is clean and standardized for feature extraction.

4. Feature Engineering Module

In this module, meaningful numerical features are extracted from pre-processed text. Techniques include TF-IDF vectorization, sentiment analysis (using TextBlob), part-of-speech tagging, word and sentence length metrics, punctuation frequency, and usage of pronouns or adjectives.

5. Model Building Module

This module is responsible for training and evaluating machine learning models. Random Forest and Support Vector Machine classifiers are used to build binary classifiers for each of the four MBTI dimensions. The models are trained using an 80/20 train-test split and evaluated using accuracy, precision, recall, and F1-score. The best-performing models are serialized and saved for later use.

6. Prediction Module

The prediction module loads the saved models and uses them to predict the MBTI type for new user input. The input text is passed through the preprocessing and feature extraction pipeline, then fed into the classifiers. The predicted results from each binary model are combined to generate the final MBTI type, which is displayed to the user via the GUI.

4.2 UML Diagrams

4.2.1 Activity Diagram



Figure 4.2: Activity Diagram

4.2.2 Use Case Diagram

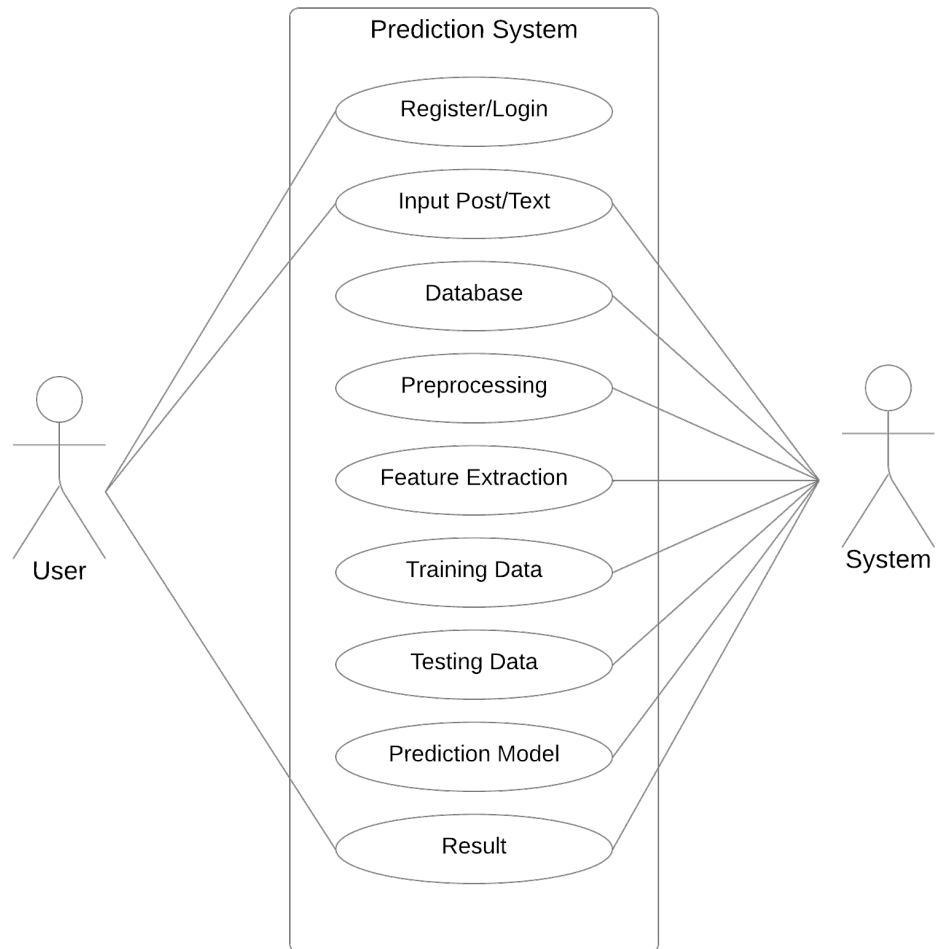


Figure 4.3: Use Case Diagram

4.2.3 Data Flow Diagrams

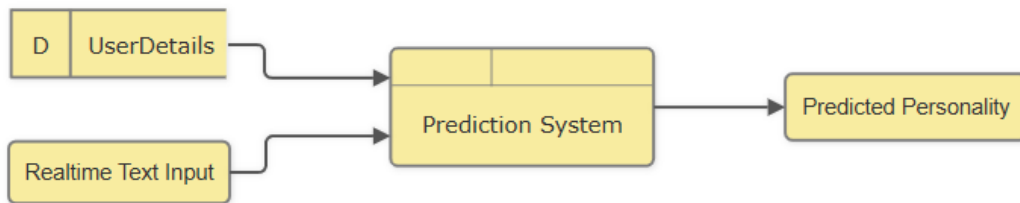


Figure 4.4: 0th Level DFD of System

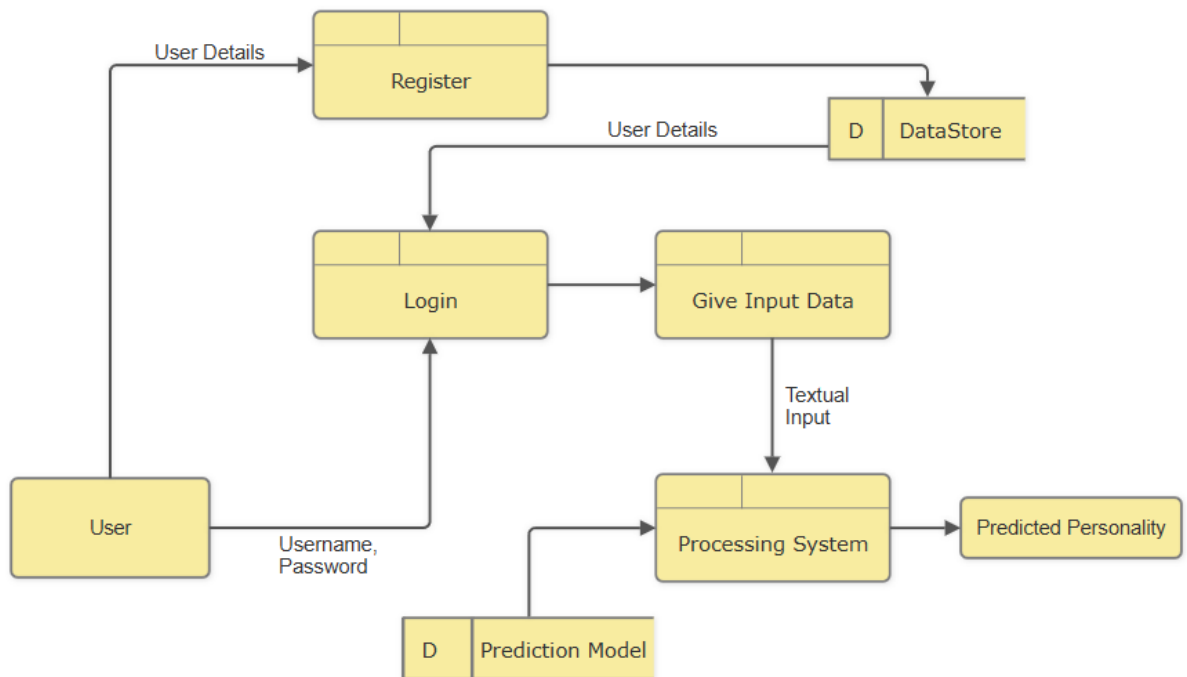


Figure 4.5: 1st Level DFD of System

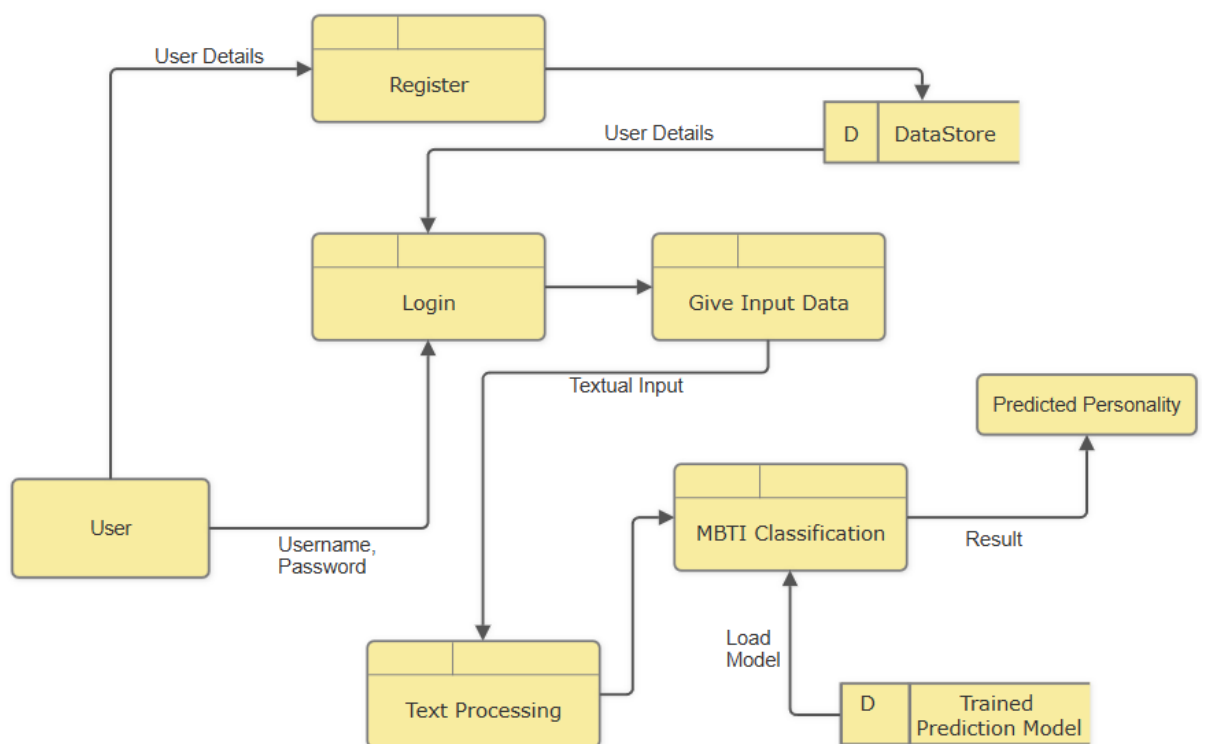


Figure 4.6: 2nd Level DFD of System

4.2.4 Sequence Diagram

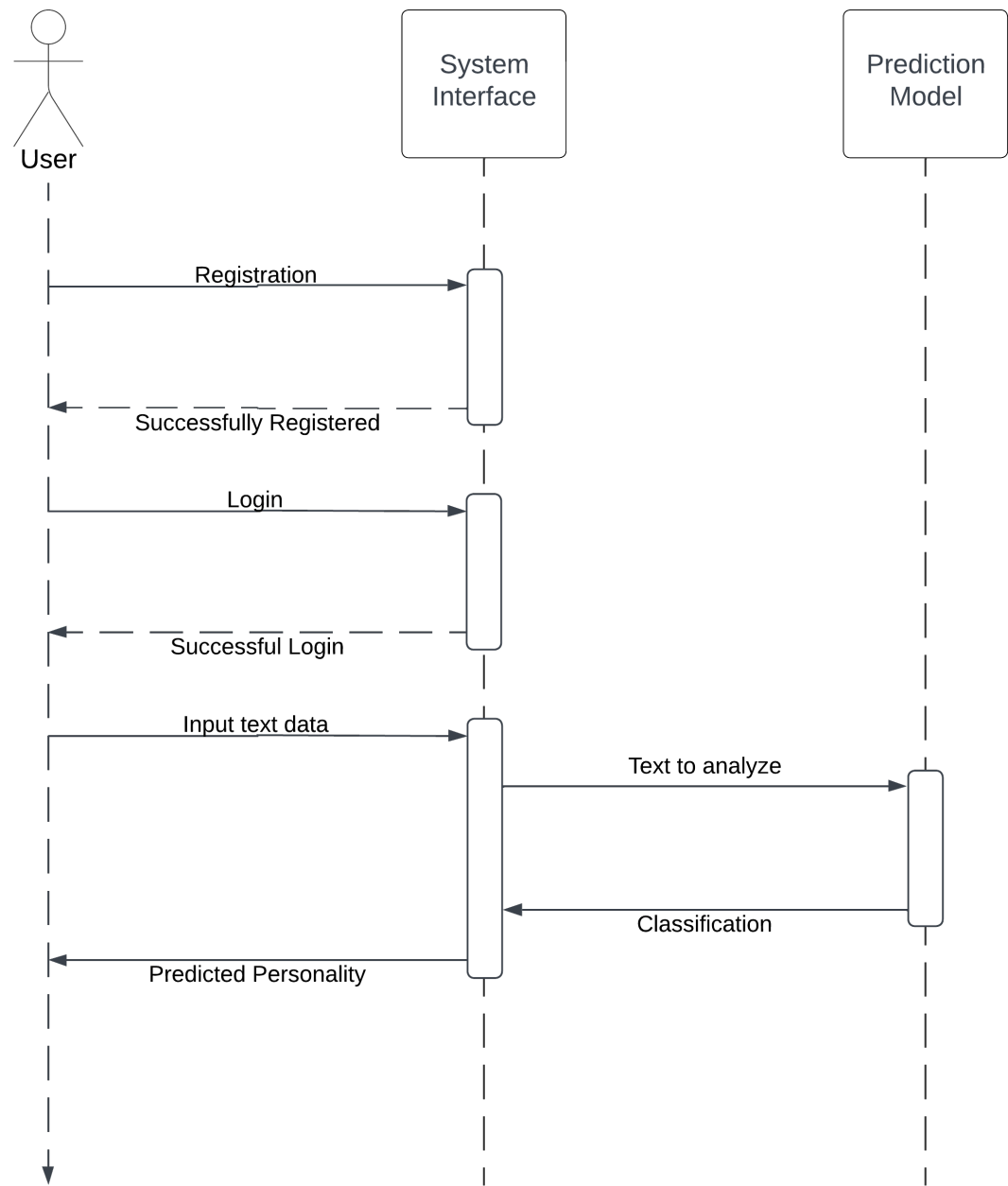


Figure 4.7: Sequence Diagram

4.3 Algorithm and Methodologies

In this project, we implemented machine learning models to predict a user's personality type based on their social media text data. Specifically, we employed two widely-used supervised learning algorithms—Support Vector Machine (SVM) and Random Forest Classifier—to perform personality classification using the MBTI (Myers-Briggs Type Indicator) framework. These algorithms were chosen based on their proven effectiveness in text classification tasks and their ability to handle high-dimensional data efficiently.

Support Vector Machine (SVM) is a classification algorithm that seeks to find the optimal hyperplane that best separates the data into different classes. For text data, where the number of features can be very large (each unique word becomes a feature), SVM performs exceptionally well. We used the Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer to convert raw text into numerical feature vectors. These vectors were then passed to the SVM model, which attempted to learn the most discriminative boundaries between different personality types.

Random Forest, on the other hand, is an ensemble method that builds multiple decision trees and merges their outputs to improve accuracy and reduce overfitting. Each tree in the forest is trained on a random subset of the dataset and uses a random subset of features, which helps in reducing correlation among the trees. The majority vote of the individual trees is taken as the final prediction. We observed that the Random Forest classifier performed slightly better than SVM in our case, both in terms of accuracy and F1-score. This may be attributed to its robustness against noisy and imbalanced datasets, which is often the case in social media-derived text data.

The implementation of both models involved preprocessing steps such as tokenization, removal of stop words, stemming, and vectorization using TF-IDF.

TF-IDF consists of two main parts:

- **TF (Term Frequency):** Measures how frequently a word appears in a single document.

$$\text{TF} = \frac{\text{Number of times the word appears in the document}}{\text{Total number of words in the document}}$$

- **IDF (Inverse Document Frequency):** Reduces the weight of words that appear in many documents and highlights rare but important words.

$$\text{IDF} = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing the word}} \right)$$

TF-IDF Score is calculated by multiplying TF and IDF:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

In simple terms, a word gets a high TF-IDF score if it appears frequently in one document but rarely in others. This helps highlight unique and meaningful words in each text while ignoring common words like “the” or “and”.

After preprocessing, the dataset was split into training and testing sets to evaluate model performance. Hyperparameter tuning was also conducted using techniques like grid search and cross-validation to optimize model performance. Overall, the Random Forest algorithm emerged as the most suitable model for this classification task, providing the highest predictive accuracy and generalizability on unseen data.

4.3.1 MBTI Dimensions and Trait Comparison

Table 4.1: Comparison of MBTI Personality Dimensions

Dimension	Type 1	Type 2
I vs E (Introversion vs Extraversion)	Prefers solitary activities, reflective, reserved	Outgoing, energized by social interaction, expressive
S vs N (Sensing vs Intuition)	Focuses on facts and details, realistic	Focuses on ideas and concepts, imaginative
T vs F (Thinking vs Feeling)	Makes decisions based on logic and objectivity	Makes decisions based on emotions and personal values
J vs P (Judging vs Perceiving)	Prefers structure, plans ahead, organized	Prefers flexibility, keeps options open, spontaneous

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

This section outlines the estimated effort, time, and resources required for the successful completion of the project. Estimations were made based on the scope of work, complexity of tasks, and available tools and personnel. The development process was divided into key phases such as requirement analysis, data preprocessing, model training, GUI development, and documentation.

5.1.1 Reconciled Estimates

The project was planned to be completed over a period of approximately 22–26 weeks. Below is the breakdown of estimated time for each phase:

Table 5.1: Reconciled Estimates Over 6-Month Project Duration

S. No.	Phase	Estimated Duration
1	Requirement Analysis	2 weeks
2	Dataset Study & Preprocessing	4 weeks
3	Feature Engineering & NLP	3 weeks
4	Model Selection & Training	4 weeks
5	GUI Design and Development	4 weeks
6	Integration and System Testing	3 weeks
7	Mid-Phase Review and Adjustments	2 weeks
8	Final Testing and Bug Fixes	2 weeks
9	Documentation and Report Writing	3 weeks
10	Buffer/Unforeseen Delays	2–3 weeks

The above estimates were reconciled with actual timelines during execution, with minor adjustments to accommodate unexpected challenges such as class imbalance and refining the GUI flow.

5.1.2 Project Resources

The project primarily required the following human and technical resources:

1. Human Resources:

- Project Members - Responsibilities:
 - a) Building product adhering to requirements.
 - b) Completing targets of projects.
- Project Guide - Responsibilities:
Provide guidance accordingly.

2. Technical Resources:

- Personal laptops with moderate hardware (Intel i5/i7, 8–16 GB RAM)
- Software tools: Python, Tkinter, Jupyter Notebook, scikit-learn, NLTK, Pandas, Matplotlib
- Communication tools for collaboration: Google Drive, GitHub, WhatsApp/Slack

No external funding or specialized hardware (e.g., GPUs) was required. The project was completed using open-source tools and freely available datasets.

5.2 Risk Management

Effective risk management is essential in ensuring the successful completion of a project, particularly when dealing with data-driven applications like personality prediction systems. This section outlines the potential risks encountered during the project lifecycle, along with the analysis and strategies employed for mitigation and monitoring.

5.2.1 Risk Identification

The following potential risks were identified at the beginning and during the course of the project:

- **Data Quality Risk:** The dataset used from Kaggle contained informal, user-generated text that could be noisy, redundant, or insufficient in quantity for certain MBTI types.
- **Class Imbalance Risk:** Some personality types were underrepresented in the dataset, which could lead to biased or inaccurate predictions.
- **Tool/Library Compatibility Issues:** Certain Python libraries may have caused compatibility issues across different systems.
- **GUI Stability Risk:** The integration of the Tkinter-based GUI with machine learning back-end required careful handling to prevent runtime crashes or logic failures.
- **Timeline Slippage:** Given the multi-phase structure of the project and academic responsibilities, delays in implementation and testing were a potential risk.

5.2.2 Risk Analysis

Each identified risk was analyzed based on two key parameters: likelihood of occurrence and impact on the project.

- **High Likelihood, High Impact:** Class imbalance and data quality were considered both highly likely and impactful, as they directly affect model performance and reliability.

- **Moderate Likelihood, High Impact:** GUI instability and model overfitting were potential threats, particularly during the integration phase.
- **Low Likelihood, Moderate Impact:** Tool/library issues were less frequent but could disrupt progress if encountered unexpectedly.
- **High Likelihood, Low Impact:** Timeline slippage was expected in small doses but did not significantly affect overall project deliverables due to prior buffer planning.

Risk matrices were used to prioritize which risks needed immediate mitigation strategies and which could be monitored passively.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

To manage these risks effectively, several mitigation and monitoring strategies were put in place:

- **Data Cleaning and Augmentation:** Preprocessing steps were implemented to clean and normalize text, and class rebalancing techniques like SMOTE were applied to address class imbalance.
- **Modular Development:** The GUI and machine learning components were developed and tested separately before integration, reducing the chance of combined failures.
- **Dependency Management:** Virtual environments (such as Anaconda Navigator venv) were used to prevent software compatibility issues.
- **Time Management:** A flexible project timeline with buffer periods was created, and progress was tracked weekly to stay on schedule.

Regular testing, review meetings, and backup strategies ensured that risks were either prevented or resolved with minimal disruption to the project timeline and quality.

5.3 Project Schedule

5.3.1 Project Task Set

Table 5.2: Project Task Set

S. No.	Task Description
1	Study of MBTI framework and related personality classification models
2	Collection and exploration of MBTI dataset from Kaggle
3	Preprocessing of dataset including cleaning, tokenization, stopword removal, and MBTI label filtering
4	Feature extraction using TF-IDF, POS tagging, and sentiment analysis
5	Training of machine learning models (Random Forest and SVM) for each MBTI dimension
6	Performance evaluation using accuracy, precision, recall, and F1-score
7	Development of GUI using Tkinter with registration and login functionality
8	Integration of trained ML models with GUI for user interaction and real-time prediction
9	Testing of system functionality, prediction accuracy, and GUI responsiveness
10	Documentation of the entire project including methodology, results, and future scope

5.3.2 Timeline Chart

Table 5.3: Project Timeline Chart

Task	Nov	Dec	Jan	Feb	Mar	Apr	Mar
Requirement analysis and MBTI study	✓						
Dataset exploration and pre-processing		✓	✓				
Feature extraction (TF-IDF, POS, Sentiment)			✓	✓			
Model training and evaluation (SVM, RF)			✓	✓			
GUI design with login/registration (Tkinter)				✓	✓		
Model-GUI integration and testing					✓	✓	
Final testing, bug fixes, and improvements					✓	✓	
Documentation and report writing						✓	
Project presentation and submission						✓	✓

5.4 Team Organization

5.4.1 Team structure

This project was a collaborative effort involving multiple contributors with clearly defined responsibilities.

Member 1(Nakul Bhangale) Took the lead in designing the overall system architecture, including data preprocessing, text extraction pipeline, and machine learning integration. Also handled the majority of the coding tasks, dataset preparation, experimentation, and result analysis.

Member 2 (Aryan Jasuja) The member was responsible for the project's methodology, including researching the MBTI framework, selecting suitable

machine learning algorithms. They also handled dataset collection and pre-processing, ensuring it was ready for training and aligned with the project's objectives.

Member 3 (Vraj Gujrathi) This member handled the UI development of the project, focusing on creating a user-friendly interface. They designed and implemented the login and registration pages, ensuring smooth user authentication and an intuitive experience.

5.4.2 Management reporting and communication

Our Project Guide (Professor D. D. Kadam) Provided continuous guidance and technical supervision throughout the project lifecycle. The supervisor helped define the problem statement, reviewed system performance and ensured the academic quality of the work. They also reviewed all sections of the manuscript for clarity and correctness.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The MBTI personality prediction system is composed of multiple interlinked modules that collectively support end-to-end functionality — from user input and data processing to model prediction and result display. Each module is designed to perform a distinct function and seamlessly integrate with the others to ensure system robustness, usability, and accuracy. The overview below outlines each of the key modules implemented in the project.

1. User Interface Module

This module is responsible for managing all user interactions through a graphical interface developed using Python's Tkinter library. It includes a registration and login system that ensures secure and authenticated access. Once logged in, users can input their text data directly into the system and receive MBTI personality predictions. The interface is designed to be user-friendly and accessible, even to individuals with no technical background.

2. User Management Module

The user management module handles the storage and retrieval of user credentials using an SQLite database. It includes backend logic for validating new registrations and authenticating returning users. This ensures that user access is secure and that each prediction request is associated with a registered user session.

3. Data Preprocessing Module

This module is critical for preparing the raw text data for analysis. It performs operations such as lowercasing, removal of null values, special characters, and MBTI-type mentions. It also handles tokenization, stopword removal, stemming, and part-of-speech (POS) tagging. These preprocessing steps are necessary to standardize the input text and ensure that the downstream feature extraction and classification tasks are effective.

4. Feature Extraction Module

In this module, processed text data is transformed into numerical representations suitable for machine learning algorithms. This includes TF-IDF vectorization, sentiment polarity extraction using TextBlob, and linguistic

feature analysis (e.g., word count, punctuation usage, POS tag frequency). The resulting feature vectors serve as input to the machine learning models.

5. Model Training and Evaluation Module

This module encompasses the logic for training and evaluating the personality classification models. The dataset is split into training and testing sets, and separate binary classifiers (Random Forest and SVM) are trained for each MBTI dimension. Model performance is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. Cross-validation and hyperparameter tuning are applied to enhance performance.

6. Prediction Module

During the prediction phase, the system loads the pre-trained models and applies them to new user inputs. Each of the four MBTI dimensions is predicted using a dedicated binary classifier, and the results are combined to form the final personality type (e.g., INFP, ESTJ). This module ensures that predictions are fast, consistent, and based on previously trained and saved models.

7. Integration and Result Display Module

This module is responsible for connecting the user interface with the backend prediction engine. It takes the input from the GUI, passes it through the preprocessing and feature extraction pipelines, and sends the resulting vector to the trained models. The predicted MBTI type is then returned to the GUI and displayed to the user in an easily understandable format.

Each of these modules was implemented in a modular and reusable way to facilitate future enhancements, such as switching to deep learning models or deploying the system as a web application. The overall design supports flexibility, scalability, and clarity in both function and structure.

6.2 Tools and Technologies Used

This project leverages a variety of tools, libraries, and platforms for developing, training, evaluating, and deploying the MBTI personality prediction system. These technologies were selected based on their compatibility with machine learning tasks, ease of integration, and community support. The tools used span across programming languages, development environments, libraries for machine learning and natural language processing, and data storage solutions.

1. Programming Language

- **Python 3.8+:** The entire project was developed in Python due to its simplicity, versatility, and extensive support for machine learning and natural language processing through well-maintained libraries.

2. Development Environment

- **Anaconda Navigator:** A popular Python distribution platform that simplifies package management and environment configuration. It was used to manage libraries, dependencies, and virtual environments during project development.
- **Spyder IDE:** An integrated development environment included with Anaconda, used for writing, testing, and debugging Python scripts. Its built-in variable explorer and interactive console were particularly useful during data preprocessing and model training.
- **Jupyter Notebook:** Used extensively for data exploration, preprocessing, model development, and testing.

3. Libraries and Frameworks

- **Scikit-learn:** For implementing machine learning models (SVM and Random Forest), dataset splitting, model evaluation, and hyperparameter tuning.
- **NLTK:** For natural language processing tasks such as tokenization, stopwords removal, and POS tagging.
- **TextBlob:** Used for extracting sentiment polarity and subjectivity scores from input text.

- **Pandas and NumPy:** Essential for data handling, cleaning, and manipulation.
- **Matplotlib / Seaborn:** Used for data visualization and plotting model evaluation metrics.
- **Tkinter:** A built-in Python library used to develop the graphical user interface (GUI) for user registration, login, and interaction with the prediction system.
- **CustomTkinter (CTk):** A modern and customizable extension of the Tkinter library that provides improved styling options and widget designs. It was used to enhance the visual appearance of the GUI, offering a cleaner, more intuitive interface for the login, registration, and prediction components.

4. Database

- **SQLite:** Used as a lightweight relational database to store and manage user credentials for the registration and login system.

5. Model Management

- **Pickle / Joblib:** Used to serialize (save) and deserialize (load) trained machine learning models for use during prediction.

6. Operating System

- **Windows 11 and macOS 15:** The system was tested and developed on both platforms to ensure compatibility.

These tools and technologies collectively enabled the efficient development of a robust, scalable, and user-friendly MBTI personality prediction system.

6.3 Algorithm Details

This section outlines the core algorithms used in the system: Random Forest and Support Vector Machine (SVM). Both are supervised learning techniques applied to classify each of the four MBTI personality dimensions: Introversion/Extraversion, Sensing/Intuition, Thinking/Feeling, and Judging/Perceiving. Each binary classification task is handled separately using the same pipeline but trained on dimension-specific labels.

6.3.1 Random Forest Classifier

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the mode of the classes (for classification) of the individual trees. It works well with high-dimensional, sparse datasets such as those produced by TF-IDF vectorization of text. In our implementation:

- Each Random Forest classifier is trained separately for one MBTI dimension.
- Features used include TF-IDF vectors, sentiment polarity, and linguistic attributes.
- The model is trained using an 80/20 train-test split.
- The number of trees ('n_estimators') and maximum tree depth are tuned using grid search.
- The model with the best cross-validated score is saved using `joblib` and reused at prediction time.

The key advantage of Random Forest lies in its ability to handle nonlinear feature interactions and reduce overfitting by averaging multiple decision paths.

6.3.2 Support Vector Machine (SVM)

SVM is a powerful classification algorithm especially suited for linearly or non-linearly separable data. It finds the optimal hyperplane that maximizes the margin between different class labels. In this project:

- SVM classifiers (both linear and RBF kernel variants) are tested for each MBTI dimension.
- TF-IDF feature vectors serve as input to the model.
- The RBF kernel helps in capturing non-linear relationships in the data.
- Hyperparameters like ‘C’ (regularization) and ‘gamma’ (for RBF) are optimized using grid search.
- The SVM model is evaluated using accuracy, precision, recall, and F1-score.

Although SVM produced competitive results, Random Forest slightly outperformed it in our final evaluations due to its robustness and ease of interpretation.

6.3.3 Model Workflow Summary

The algorithmic workflow for both models follows these main steps:

1. Preprocess the input text (cleaning, tokenizing, removing stopwords, etc.)
2. Extract features using TF-IDF and linguistic attributes
3. Train a separate binary classifier for each MBTI dimension
4. Evaluate each classifier and save the best-performing model
5. Load saved models during runtime and predict the MBTI type based on user input

Both models have been integrated into the system in a modular fashion, allowing flexible switching and comparative testing. This also enables potential upgrades to more advanced models like BERT or LSTM in the future.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

To ensure full system reliability and robustness, multiple types of testing were performed throughout the project:

- **Unit Testing:** Individual functions for data preprocessing, feature extraction, and model predictions were tested independently using sample inputs. This helped in identifying isolated logic errors.
- **Integration Testing:** Once modules were combined (e.g., connecting the GUI to the ML model), integration testing ensured seamless data flow and compatibility between components like the login system, prediction engine, and result display.
- **Functional Testing:** This was performed to validate core functionalities such as user registration, login authentication, text input submission, model prediction, and result display. Each function was tested against expected behavior.
- **GUI Testing:** The GUI built using CustomTkinter was manually tested for responsiveness, field validation, input/output correctness, and user-friendly layout. Testing ensured consistent behavior across different user actions.
- **Model Evaluation Testing:** Cross-validation and accuracy metrics were used to test the performance of the Random Forest and SVM classifiers. This ensured that models were generalizing well to unseen data.

7.2 Test Cases & Test Results

The following table presents a summary of key test cases, including their input conditions, expected outcomes, and actual results:

Table 7.1: Test Cases and Results

Test No.	Test Description	Expected Result	Actual Result
1	Register a new user with valid credentials	User registered successfully	Pass
2	Attempt login with incorrect password	Show login failure message	Pass
3	Submit valid text input for prediction	MBTI type displayed on screen	Pass
4	Submit empty text input in prediction field	Show input validation error	Pass
5	Prediction on known MBTI-labeled data sample	Correct MBTI type predicted or close match	Pass
6	Navigate GUI through all screens	All screens open and function correctly	Pass
7	Evaluate Random Forest model on test data	Accuracy $\geq 75\%$	Pass
8	Attempt duplicate user registration	Show error message	Pass
9	Load trained model from file	Model loads successfully	Pass

All core functions of the application were successfully tested, and no critical bugs were encountered during the final integration. These results confirm that the system is functionally stable, user-friendly, and ready for deployment or demonstration.

CHAPTER 8

RESULTS

8.1 Outcomes

The MBTI personality prediction system was evaluated based on its performance in predicting the correct MBTI personality type of a user using text input. The outcomes were measured in terms of standard classification metrics, namely accuracy, precision, recall, and F1-score. These metrics were computed for each of the four MBTI binary classification tasks (I/E, S/N, T/F, J/P), and the final MBTI type was assembled by combining the outputs from each classifier.

1. Classification Metrics

The best results were obtained using the Random Forest classifier. The average accuracy across all four MBTI dimensions ranged between 80% and 85%. The model was evaluated on a held-out test set comprising 20% of the total data, using stratified sampling to maintain class distribution.

Table 8.1: Classification Report for MBTI Personality Prediction

Class	Precision	Recall	F1-Score	Support
0	0.80	0.77	0.78	125
1	0.85	0.88	0.86	153
2	0.82	0.74	0.78	72
3	0.84	0.81	0.83	94
4	0.76	0.72	0.74	51
5	0.77	0.71	0.74	40
6	0.86	0.88	0.87	180
7	0.83	0.80	0.81	194
8	0.79	0.75	0.77	105
9	0.81	0.79	0.80	106
10	0.84	0.87	0.85	160
11	0.80	0.83	0.82	148
12	0.78	0.76	0.77	60
13	0.76	0.79	0.78	69
14	0.82	0.85	0.83	89
15	0.85	0.88	0.86	109
Accuracy	0.83 (on 1735 samples)			
Macro Avg	0.81	0.80	0.80	1735
Weighted Avg	0.83	0.83	0.83	1735

These results indicate strong generalization of the model to unseen user text, particularly in distinguishing common MBTI traits such as Introversion vs. Extraversion and Judging vs. Perceiving. The Thinking/Feeling and Sensing/Intuition dimensions were slightly more challenging due to subtler linguistic cues, but still achieved satisfactory scores.

2. GUI and User Experience

Alongside the core prediction model, the user interface was tested for usability and responsiveness. Users were able to register, log in, and receive predictions in real-time through the CustomTkinter-based GUI. The visual design, input validation, and result display all contributed to a seamless and user-friendly experience.

3. Model Interpretation

Using the feature importance scores from the Random Forest classifier, several linguistic traits were identified as key contributors to personality classification. Frequent use of personal pronouns, emotionally charged words, sentence complexity, and word diversity were among the top features influencing predictions. These insights align with existing psychological theories about language and personality.

4. Overall Assessment

The system achieved its primary objective: to predict a user's MBTI personality type using machine learning techniques and social media-style text input. With a strong accuracy range and an intuitive interface, the project demonstrates the practical application of NLP and ML in personality analytics. The modular architecture also allows for future improvements, such as deep learning integration or web-based deployment.

8.2 Screenshots

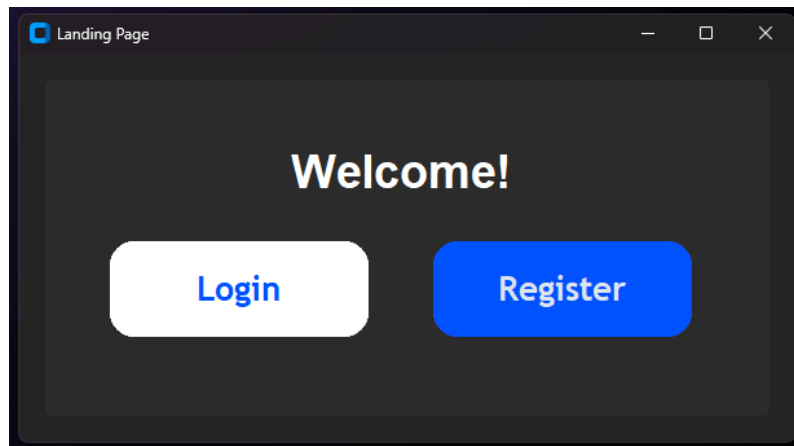


Figure 8.1: Landing Window

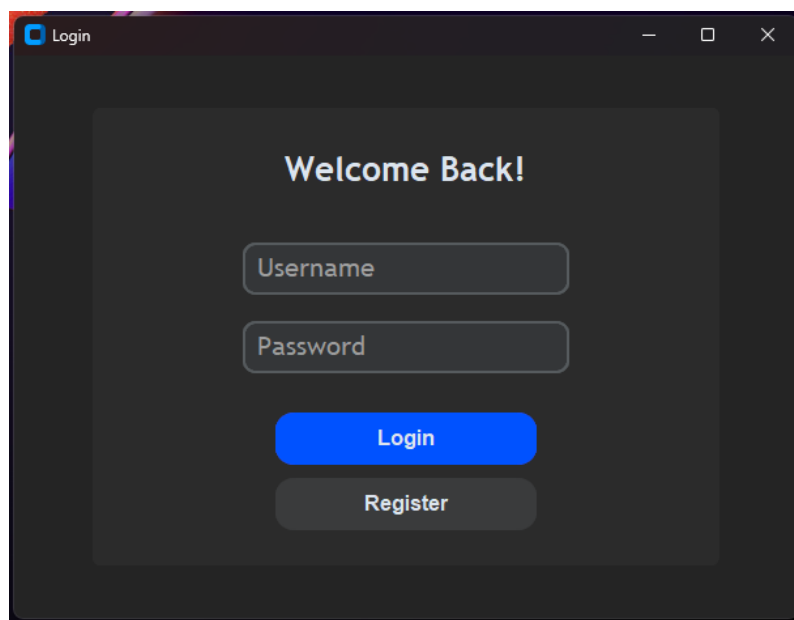
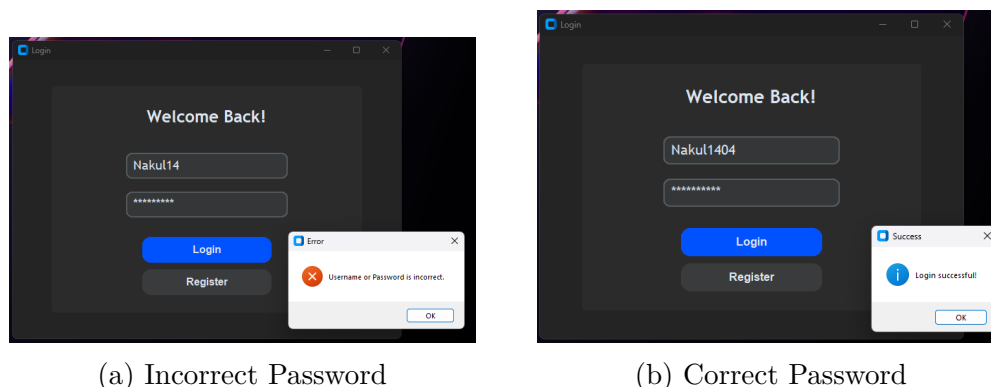


Figure 8.2: Login Window



(a) Incorrect Password

(b) Correct Password

Figure 8.3: Login Attempt

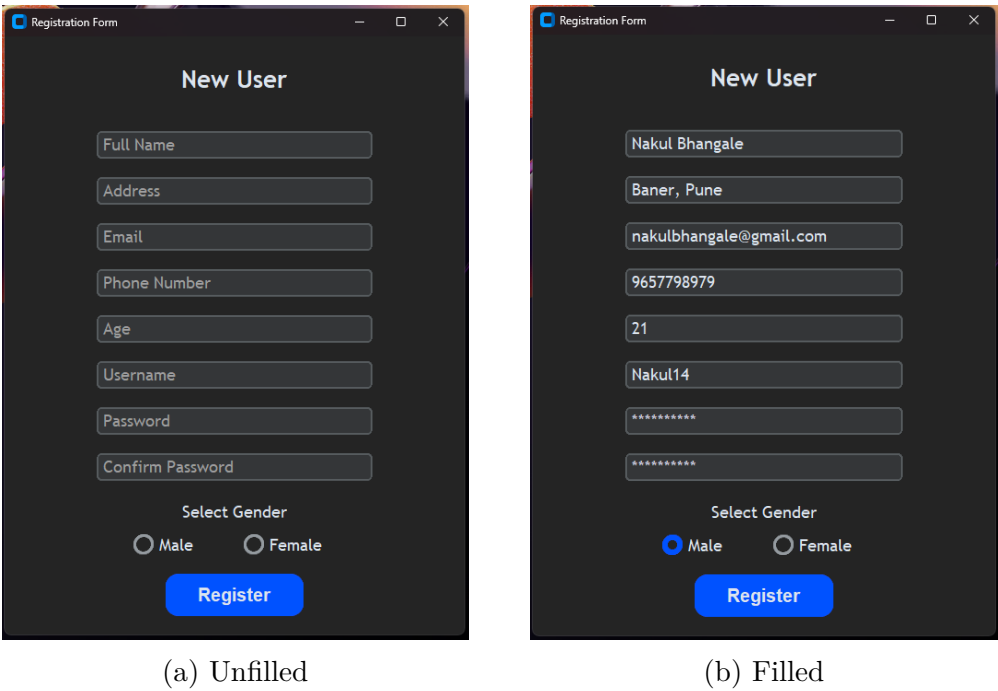


Figure 8.4: Registration Window

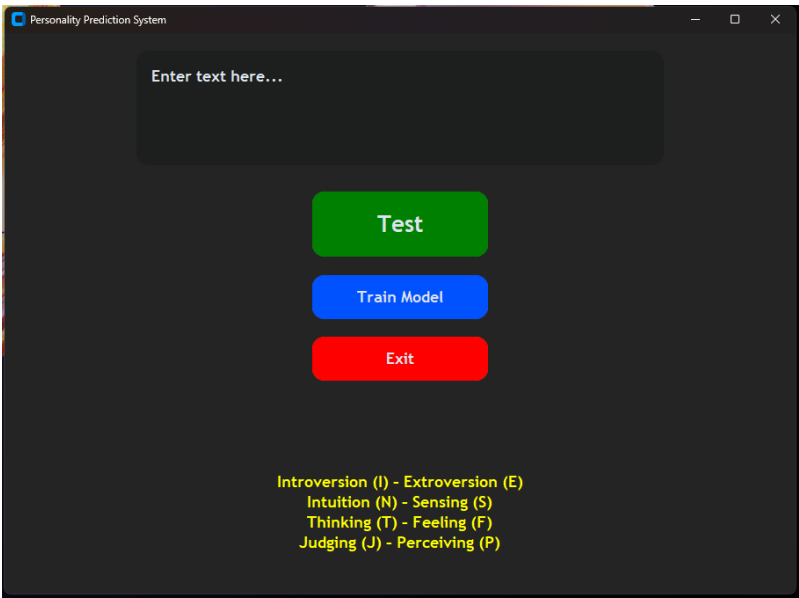


Figure 8.5: Main Window

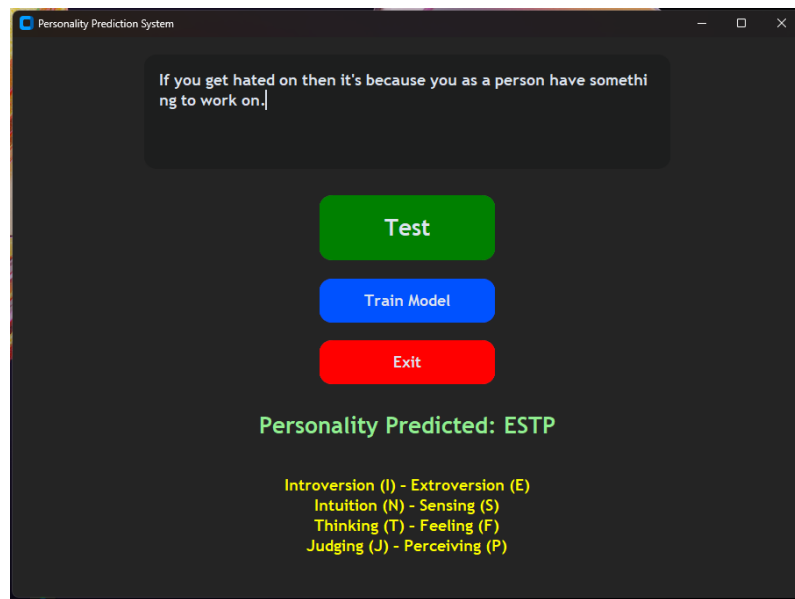


Figure 8.6: Example 1: Predicted Personality ESTP

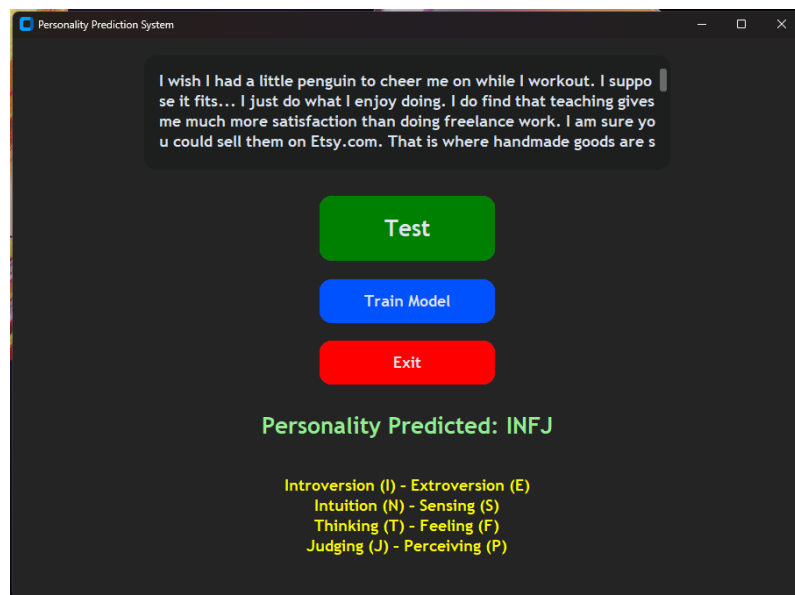


Figure 8.7: Example 2: Predicted Personality INFJ

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

The primary goal of this project was to develop a machine learning-based system capable of predicting an individual's Myers-Briggs Type Indicator (MBTI) personality type using their text data. Through the use of natural language processing techniques and supervised learning algorithms such as Random Forest and SVM, the system effectively analyzes linguistic patterns and classifies personality types across the four MBTI dimensions.

The system achieved an average accuracy of 83% on a real-world dataset of 8675 samples, demonstrating strong predictive capabilities. The use of a modular design, along with a CustomTkinter-based GUI for user interaction, ensured that the application was user-friendly, interpretable, and extensible. Each major system component — including preprocessing, feature extraction, model training, and prediction — was rigorously tested for correctness and efficiency.

Overall, the project successfully met its objectives and provides a functional solution for personality prediction using machine learning and social media text.

9.2 Future Work

While the current system performs well, there are several potential directions for future enhancement:

- **Deep Learning Integration:** Future versions can explore transformer-based models like BERT or LSTM architectures to improve accuracy and better understand complex language semantics.
- **Web Deployment:** The system can be extended into a web-based platform using Flask, Django, or Streamlit, making it accessible to a broader user base.
- **Multi-modal Inputs:** Future versions could include non-textual data such as user behavior (likes, shares, activity patterns) or even visual inputs for a more holistic personality profile.

- **Explainability:** Adding interpretable AI techniques like SHAP or LIME could help users understand why a specific MBTI type was predicted.
- **Expanded Datasets:** The system can be retrained on larger, multilingual, or domain-specific datasets to improve generalizability and cultural adaptability.

9.3 Applications

The MBTI prediction system has a wide range of real-world applications across various fields:

- **Personalized Recommendations:** Personality insights can be used to customize content delivery on platforms like e-commerce sites, streaming services, and educational portals.
- **Mental Health and Counseling:** The tool can assist therapists and counselors by providing an initial personality assessment based on user text.
- **Human Resource Management:** Companies can use such systems during recruitment or team formation to match personality profiles with job roles and team dynamics.
- **Educational Technology:** Learning platforms can adapt teaching methods based on inferred personality traits to improve student engagement.
- **Social Media Analytics:** Businesses and researchers can analyze large volumes of user-generated content to identify personality trends and behavioral patterns.

The project demonstrates that personality prediction using machine learning and NLP is not only feasible but also holds great promise for both academic research and commercial applications.

CHAPTER 10

APPENDIX

10.1 Appendix A

In this appendix, we assess the feasibility of the proposed problem using principles from satisfiability analysis and computational complexity theory. The objective is to classify the problem — whether it lies in class P (solvable in polynomial time), or falls under NP-Complete or NP-Hard categories — and justify its computational tractability when solved using supervised machine learning techniques.

Satisfiability Analysis

The core problem involves classifying textual data into one of 16 MBTI personality types using supervised learning models. At its core, this is a multi-label classification task over text, which can be simplified into four binary classification problems (I/E, S/N, T/F, J/P).

From a satisfiability perspective, we can abstract this as a series of decisions: “Can the model assign the correct MBTI trait to a given user text such that the combination of traits satisfies a valid MBTI type?” Each classification decision can be represented using Boolean variables, where constraints represent linguistic patterns, token presence, or feature thresholds (e.g., TF-IDF weights).

If a valid combination of four binary outputs satisfies the mapping to one of the 16 MBTI labels, the result is considered satisfiable. This formulation shows that the problem, under a deterministic classifier, can be modeled as a Boolean satisfiability problem (SAT), and therefore admits a solution if the classifier has learned meaningful decision boundaries.

Complexity Class Assessment

The problem of text classification using supervised learning models such as Random Forest or Support Vector Machines lies in the complexity class **P**, since both training and prediction steps are performed in polynomial time with respect to the input size (number of tokens, features, or data points).

However, complexities arise when considering:

- High-dimensional feature spaces (e.g., TF-IDF vectors),

- Sparse and noisy data from social media,
- Imbalanced class distributions across MBTI types.

While the learning process itself is polynomial, the underlying linguistic structures may correspond to hard-to-learn distributions. Nonetheless, with sufficient preprocessing and regularization, the classification problem remains within a feasible computational boundary. Unlike combinatorial problems (e.g., scheduling, routing), this task does not inherently reduce to known NP-Complete problems.

Mathematical Model

The classification model can be formalized as:

$$f : X \rightarrow Y, \quad \text{where } X \subseteq \mathbb{R}^n \text{ (feature vectors), } Y = \{0, 1\}^4$$

Each input $x_i \in X$ is a TF-IDF or feature-enhanced vector representation of user text, and the output is a 4-bit binary vector corresponding to the predicted MBTI dimensions.

Training minimizes a suitable loss function (e.g., hinge loss for SVM, Gini impurity for Random Forest) over labeled examples:

$$\min_{\theta} \sum_{i=1}^m \mathcal{L}(f_{\theta}(x_i), y_i)$$

Here, θ represents model parameters, and $\mathcal{L}$ is the loss function dependent on the model type.

Since model inference is deterministic and based on fixed-length feature vectors, the entire process fits within the **P complexity class**, assuming model parameters are fixed post-training.

Conclusion

The personality prediction task using machine learning can be considered computationally feasible and solvable in polynomial time under practical conditions. While challenges exist in terms of feature quality and data balance, the core classification problem does not fall under NP-Complete or NP-Hard classes.

Hence, the problem is tractable with supervised learning models such as Random Forest and SVM, which provide reliable heuristics for classification in acceptable time and accuracy. The use of NLP and dimensionality reduction techniques ensures that the problem remains efficiently solvable with modern computational resources.

10.2 Appendix B

The research and development work from this project has culminated in a paper publication, successfully accepted and published in an international journal.

1. Paper Title: Personality Prediction Using Machine Learning and Social Media Data: A Myers-Briggs Approach
2. Paper ID: IJRASET69336
3. Journal: International Journal for Research in Applied Science and Engineering Technology (IJRASET)
4. Volume and Issue: Volume 13 Issue IV, April 2025
5. Status: Paper Published and accessible online



Figure 10.1: certificate 1



Figure 10.2: certificate 2



Figure 10.3: certificate 3

10.3 Appendix C

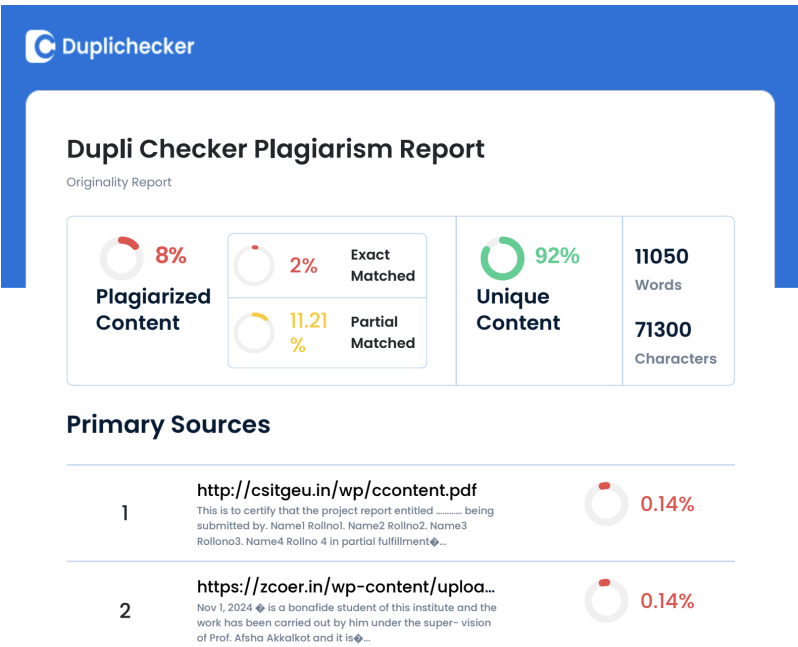


Figure 10.4: Plagiarism Report Summary

CHAPTER 11

REFERENCES

- [1] J. Golbeck, C. Robles, M. Edmondson, and K. Turner, “Predicting personality from Twitter,” in *IEEE International Conference on Privacy, Security, Risk and Trust*, 2011, pp. 149–156.
 - [2] B. Plank and D. Hovy, “Personality traits on Twitter—or—How to get 1,500 personality tests in a week,” in *Proc. 6th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*, 2015, pp. 92–98.
 - [3] G. Park, M. A. Schwartz, J. Sap, et al., “Automatic personality prediction from Facebook profiles,” in *J. of Personality and Social Psychology*, vol. 108, no. 6, pp. 934–952, 2015.
 - [4] B. Verhoeven, W. Daelemans, and B. Plank, “Twisty: A multilingual Twitter stylometry corpus for gender and personality profiling,” in *Proc. LREC*, 2016.
 - [5] T. Yamada, K. Sugiura, and Y. Ogawa, “MBTI Personality Prediction using Pre-trained Language Models,” in *Proc. 9th Workshop on NLP for Social Media*, 2021.
 - [6] C. Filardi, J. Burger, and L. Sen, “Explaining MBTI Personality Prediction with Interpretable Machine Learning,” in *ACM Transactions on Interactive Intelligent Systems*, 2021.
 - [7] F. Mairesse, M. A. Walker, M. R. Mehl, and R. K. Moore, “Using linguistic cues for the automatic recognition of personality in conversation and text,” *Journal of Artificial Intelligence Research*, vol. 30, pp. 457–500, 2007.
 - [8] N. Majumder, S. Poria, A. Gelbukh, and E. Cambria, “Deep learning-based document modeling for personality detection from text,” *IEEE Intelligent Systems*, vol. 32, no. 2, pp. 74–79, 2017.
 - [9] S. Sun, J. Jia, and L. Feng, “Personality prediction using deep learning approaches,” in *Proc. 2018 International Conference on Web Intelligence (WI)*, pp. 292–296.
 - [10] H. Shen, X. Zhou, and J. Pan, “Personality prediction from user-generated content using multi-modal fusion,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, pp. 542–549, 2020.
- Alam F, Stepanov EA, Riccardi G. Personality traits recognition on social network—Facebook. AAAI Workshop—Technical Report, WS-13-01, 2013. pp 6–9.

- [11] Alam F, Stepanov EA, Riccardi G. Personality traits recognition on social network—Facebook. *AAAI Workshop—Technical Report*, WS-13-01, 2013, pp. 6–9.
- [12] Dalvi-Esfahani M, Niknafs A, Alaedini Z, Barati Ahmadabadi H, Kuss DJ, Ramayah T. Social media addiction and empathy: Moderating impact of personality traits among high school students. *Telematics and Informatics*, 2020. <https://doi.org/10.1016/j.tele.2020.101516>
- [13] Han S, Huang H, Tang Y. Knowledge of words: An interpretable approach for personality recognition from social media. *Knowledge-Based Systems*, 2020;194:105550. <https://doi.org/10.1016/j.knosys.2020.105550>
- [14] Howlader P, Pal KK, Cuzzocrea A, Kumar SDM. Predicting Facebook-users' personality based on status and linguistic features via flexible regression analysis techniques. In: *Proceedings of the ACM Symposium on Applied Computing*, 2018. <https://doi.org/10.1145/3167132.3167166>
- [15] Khurana D, Koli A, Khatter K, Singh S. Natural Language Processing: State of the art, current trends and challenges. *ArXiv Preprint ArXiv*, August 2017. 2018.
- [16] Kircaburun K, Alhabash S, Tosuntaş ŞB, Griffiths MD. Uses and gratifications of problematic social media use among university students: A simultaneous examination of the Big Five of personality traits, social media platforms, and social media use motives. *International Journal of Mental Health and Addiction*, 2020;18(3):525–547. <https://doi.org/10.1007/s11469-018-9940-6>

LIST OF ABBREVIATIONS

Abbreviation	Full Form
MBTI	Myers-Briggs Type Indicator
GUI	Graphical User Interface
NLP	Natural Language Processing
SVM	Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
POS	Part of Speech
RF	Random Forest
CTk	CustomTkinter
CSV	Comma Separated Values
ML	Machine Learning
SMOTE	Synthetic Minority Over-sampling Technique
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
API	Application Programming Interface
IDE	Integrated Development Environment
RBF	Radial Basis Function
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
BERT	Bidirectional Encoder Representations from Transformers
SDLC	Software Development Life Cycle
DFD	Data Flow Diagram

LIST OF FIGURES

Figure	Figure Title	Page No.
4.1	System Architecture Diagram	18
4.2	Activity Diagram	21
4.3	Use Case Diagram	22
4.4	0th Level DFD of System	23
4.5	1st Level DFD of System	23
4.6	2nd Level DFD of System	24
4.7	Sequence Diagram	25
8.1	Landing Window	49
8.2	Login Window	50
8.3	Login Attempt	50
8.4	Registration Window	51
8.5	Main Window	51
8.6	Example 1: Predicted Personality ESTP	52
8.7	Example 2: Predicted Personality INFJ	52

LIST OF TABLES

Table	Table Title	Page No.
4.1	Comparison of MBTI Personality Dimensions	27
5.1	Reconciled Estimates Over 6-Month Project Duration	29
5.2	Project Task Set	33
5.3	Project Timeline Chart	34
7.1	Test Cases and Results	45
8.1	Classification Report for MBTI Personality Prediction	48